ML-Based Creative App Project

<Team 3>

20231005 Siyun Kim
20231099 Taeyoon Lee
20240475 Aron Martin Visser

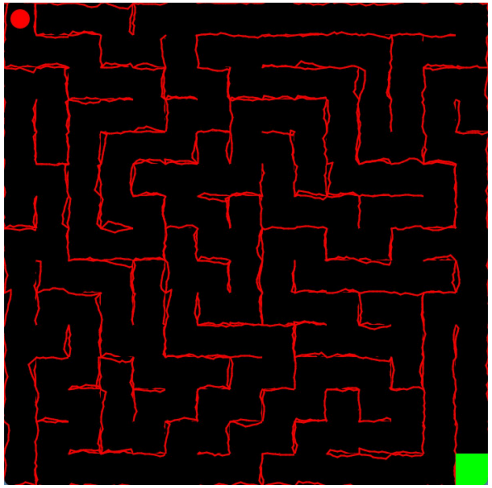
How did we get this idea

<The process through which our team came up with the idea...>

1. This game is interesting!
2. Hmm... How about making a game with a similar roaming feeling using a smartphone?
3. It might be a bit challenging to create something like *Pac-Man*, though... Then how about making a maze escape game instead?
4. Yeah! We could place a ball inside the maze and tilt the smartphone to move the ball in that direction!
5. Great! Sounds like a good idea! Now, let’s see... what else could we add to make it better?

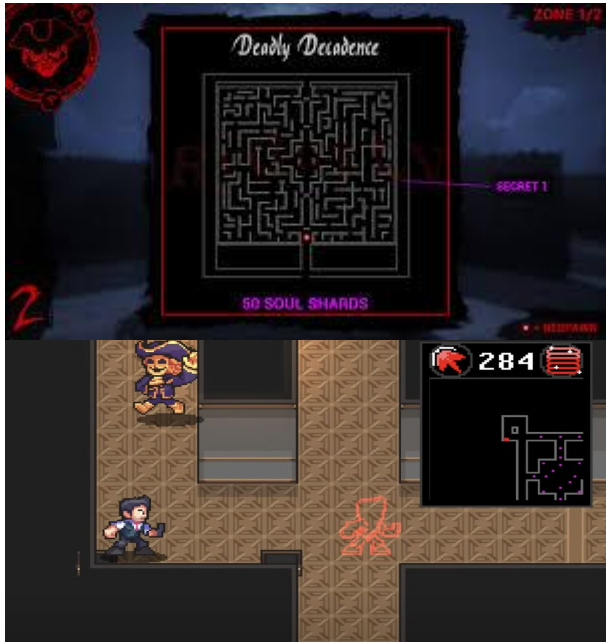
Visual Concept

For the visualization of our project, we drew inspiration from maze-like games or puzzles.

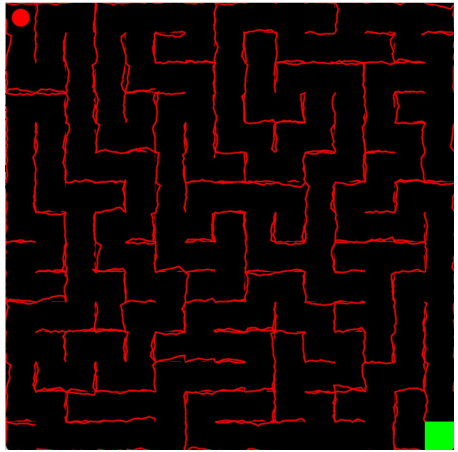


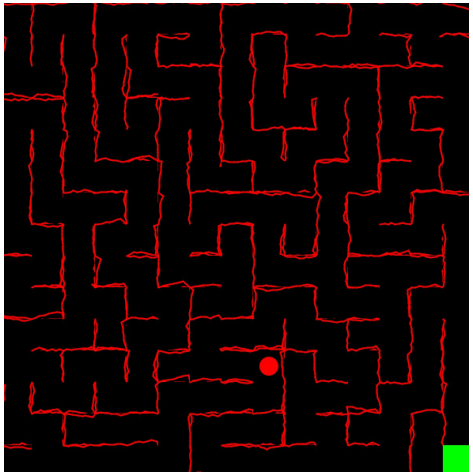
Jagged walls, red and black color scheme.

Clear goal with green.



Features





1. **Maze Navigation**:
   - Players must carefully tilt their phone to guide the ball through a complex maze.
2. **Power-Ups**:
   - Shake the phone to trigger a "teleport".
3. **Dynamic Difficulty**:
   - The maze changes when the ball arrives to the finishing point or player throws his/her phone.

Algorithms

Main Components of the Code

The code consists of the following key components:

1. Key Variables

- **`tileSize`, `gridColumns`, `gridRows`**:
  - Define the tile size and the grid's number of rows and columns.
- **`mazeGrid`**:
  - A 2D array of `Tile` objects that represents the maze structure.
- **`ball`**:
  - An instance of the `Ball` class, which represents the player's controllable ball.
- **`startColumn`, `startRow`, `goalColumn`, `goalRow`**:
  - Define the maze's starting point and goal position.
- **`teleportHistory`, `destroyHistory`**:
  - Track smartphone motion data for teleportation, and game reset actions.
  - If all data in the history array is same as ‘2’, then the program detects each as ‘teleport’ and ‘reset’.

2. Key Classes

- **`Tile` Class**:
  - Represents each tile in the maze.
  - Contains wall information (`walls`) and a `visited` flag for maze generation.
- **`Ball` Class**:
  - Handles the ball's position, movement, and teleportation logic.

3. Key Functions

- **`setup()`**:
  - Initializes the game, sets up OSC communication, and creates the maze.
- **`draw()`**:
  - The main game loop that renders the maze, ball, and handles the goal check.
- **`resetGame()`**:
  - Resets the game state and generates a new maze.
- **`buildMaze()`**:
  - Creates a random maze using a depth-first search (DFS) algorithm.
- **`oscEvent()`**:
  - Processes OSC messages from the smartphone for movement, teleportation, and resetting.
- **`highlightGoal()`**:
  - Highlights the goal tiles.
- **`displayGameOverScreen() & displayGameClearScreen()`**:
  - Displays the game-over & game-clear screen with a restart button.

Algorithms

1. Maze Generation Algorithm (buildMaze)

- **Purpose**: Generate a random maze using the depth-first search (DFS) algorithm.
- **How It Works**:
  1. Start at the Tile defined by startColumn and startRow.
  2. Mark the current tile as visited and push it onto a stack.
  3. While the stack is not empty:
     - Peek at the top tile on the stack.
     - Find a random unvisited neighbor of the current tile.
     - If a neighbor exists:
       - Remove the wall between the current tile and the neighbor.
       - Mark the neighbor as visited and push it onto the stack.
     - If no neighbors exist, pop the tile from the stack (backtracking).

2. Smartphone Motion Processing (oscEvent)

- **Purpose**: Handle OSC messages for controlling the ball's movement, teleportation, and resetting.
- **How It Works**:
  1. Receive OSC messages with direction, teleport, and reset data.
  2. Add the data to their respective history arrays (movementHistory, etc.).
  3. Use smoothing functions like getMostFrequentMovement() to reduce noise.
  4. If conditions are met:
     - Move the ball based on the smoothed direction.
     - Teleport the ball if the teleport condition is met.
     - Reset the game if the reset condition is satisfied.

3. Ball Movement Algorithm (Ball.move)

- **Purpose**: Move the ball within the maze based on the player's input.
- **How It Works**:
  1. Determine the next position based on the input direction.
  2. Check if there is a wall in the desired direction.
  3. If there is no wall, update the ball's position.

Algorithms

Process Flow

1. **Game Initialization (setup)**:
    - Set up OSC communication.
    - Generate the maze and initialize the ball.
2. **Game Loop (draw)**:
    - Render the maze and ball.
    - Check if the ball has reached the goal.
3. **OSC Message Handling (oscEvent)**:
    - Process smartphone input for movement, teleportation, and resetting.
4. **Maze Generation (buildMaze)**:
    - Generate a new maze using DFS when the game resets.
5. **Game State Reset (resetGame)**:
    - Reset all game variables and start a new maze.

Thank You For Listening!!